Práctica 1: Compilación de lenguajes de alto nivel en procesadores RISC

Asignatura: Arquitectura de Computadores
Grado en Ingeniería Informática - Ingeniería del Software
Departamento de Arquitectura de Computadores - Universidad de Sevilla

1. Objetivos

En esta práctica se estudia el repertorio de instrucciones y la programación a bajo nivel (lenguaje ensamblador) de una arquitectura moderna y abierta de tipo RISC (la arquitectura **RISC-V**). Para ello, utilizaremos el simulador **RARS** (*RISC-V Assembler and Runtime Simulator*), software de libre distribución que puede descargarse de la siguiente dirección:

https://github.com/TheThirdOne/rars

Con esta práctica se pretende que el alumno:

- Consolide sus conocimientos sobre procesadores RISC.
- Se familiarice con la arquitectura y el juego de instrucciones (ISA) de **RISC-V**.
- Sea capaz de utilizar un entorno de desarrollo para programar un procesador a bajo nivel.
- Comprenda los detalles de la ejecución del código en un procesador RISC.
- Adquiera habilidades de programación a bajo nivel de un procesador de tipo RISC.
- Conozca cómo se traducen a bajo nivel las estructuras típicas de un lenguaje de programación de alto nivel.

Es importante conocer la programación a bajo nivel (lenguaje ensamblador), ya que cualquier programa escrito en cualquier lenguaje de programación de alto nivel es traducido a lenguaje máquina (bajo nivel) por el compilador y este es el código que realmente se ejecuta. El conocimiento de cómo se traducen las estructuras de los lenguajes de alto nivel puede ayudar, por ejemplo, a hacer que el código sea más eficiente.

2. Contexto Industrial: La Arquitectura RISC-V en la Actualidad

Aunque el origen de RISC-V es académico, su adopción industrial ha crecido exponencialmente gracias a su naturaleza abierta y modular. En la actualidad, esta arquitectura no solo se utiliza para la docencia, sino que impulsa desarrollos tecnológicos en dos sectores clave del mercado: la Computación de Alto Rendimiento (HPC) y los sistemas empotrados (Microcontroladores).

Computación de Alto Rendimiento (HPC)

En el extremo de mayor potencia de cálculo, RISC-V se ha consolidado como una alternativa viable a las arquitecturas propietarias tradicionales (x86 y ARM) para centros de datos y servidores.

- **Ventana Micro Systems:** Ha desarrollado la familia de procesadores *Veyron*, basada en una arquitectura de *chiplets* que permite escalar hasta 192 núcleos por servidor, ofreciendo un alto rendimiento para cargas de trabajo intensivas en la nube.
- **SiFive:** A través de su serie *Performance* (P600/P800), implementa núcleos con ejecución fuera de orden (*out-of-order execution*) y extensiones vectoriales avanzadas, diseñadas para competir en velocidad y eficiencia en tareas de Inteligencia Artificial y procesamiento de datos masivos.

Sistemas Empotrados y Microcontroladores

En el extremo opuesto, priorizando el bajo consumo y la eficiencia de costes, RISC-V está reemplazando a arquitecturas antiguas en el diseño de microcontroladores y dispositivos IoT.

- **Espressif Systems:** Conocida por sus soluciones de conectividad, ha migrado gran parte de su catálogo a RISC-V con la serie **ESP32-C3** y posteriores. Estos SoCs (*System on Chip*) integran WiFi y Bluetooth sobre un núcleo RISC-V de 32 bits, convirtiéndose en un estándar de facto en la industria del IoT.
- **Western Digital:** Ha desarrollado los núcleos *SweRV*, procesadores de código abierto optimizados para gestionar el flujo de datos interno en discos duros y unidades de estado sólido (SSD), demostrando la capacidad de RISC-V para manejar tareas críticas de control a muy bajo nivel.

El Ecosistema en España: Barcelona Supercomputing Center (BSC)

En el ámbito europeo, España lidera el desarrollo de hardware de código abierto gracias al proyecto del **BSC-CNS**, cuyo objetivo es diseñar procesadores europeos para reducir la dependencia tecnológica externa.

- **Procesadores Lagarto:** Fue el primer hito del proyecto, demostrando la capacidad de diseñar y fabricar en silicio (65 nm) un núcleo compatible con la ISA RISC-V

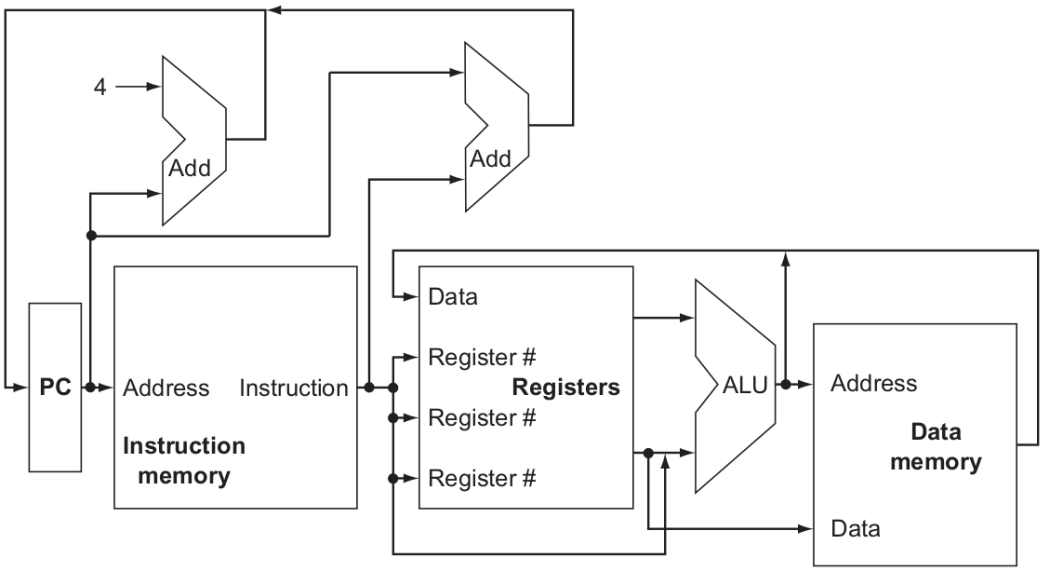
íntegramente desde el ámbito académico español.

- **Nuevas generaciones y alto rendimiento:** Tras el éxito inicial, se han desarrollado sucesores como **Sargantana** (que rompe la barrera del GHz) y diseños más recientes como **Cinco Ranch** y aceleradores vectoriales, orientados a soportar las futuras cargas de trabajo de los superordenadores europeos.

Estos ejemplos ilustran la versatilidad del conjunto de instrucciones que se estudiará en las siguientes sesiones, validando su relevancia tanto en el diseño de supercomputadores como en dispositivos de uso cotidiano.

3. Introducción a la arquitectura RISC-V

La arquitectura **RISC-V** (concretamente la variante RV32) define un procesador RISC de 32 bits, cuyo desarrollo comenzó en 2010 en la Universidad de California, Berkeley. Este diseño representa la evolución moderna y abierta de la filosofía RISC.



Este procesador se caracteriza por una arquitectura totalmente **modular**. A diferencia de los diseños monolíticos tradicionales, un procesador RISC-V estándar para propósitos generales (como el que simularemos) está compuesto fundamentalmente por:

- **El Núcleo Base de Enteros (RV32I):** Contiene los 32 registros de propósito general (x0-x31) y ejecuta las instrucciones fundamentales (aritméticas, lógicas, saltos y acceso a memoria). Es la única parte obligatoria e imprescindible de la arquitectura.
- **Las Extensiones Estándar:** Módulos opcionales que añaden funcionalidad específica al hardware. Las más comunes son la extensión **'M'** (para multiplicación y división entera por hardware) y las extensiones **'F' y 'D'** (para operaciones en coma flotante de precisión simple y doble).

- **Registros de Control y Estado (CSRs):** Un banco de registros especiales integrados en el núcleo (independientes de los registros x0-x31) encargados de gestionar el nivel de privilegio, las excepciones, las interrupciones y la configuración del sistema.

Registros:

Dentro de la CPU (en la variante RV32I) disponemos de 32 registros de números enteros de 32 bits cada uno, denominados `x0` hasta `x31`.

El registro `x0` es especial, ya que está cableado por hardware (*hardwired*) al valor 0. Cualquier lectura devuelve 0 y cualquier escritura en él es ignorada.

Aunque la arquitectura no impone restricciones hardware sobre el uso de la mayoría de estos registros (del `x1` al `x31`), existe una convención de uso establecida por la ABI (*Application Binary Interface*) que deben seguir todos los programas. Si el programador no respeta esta convención, el código resultante podría no funcionar correctamente al interactuar con el sistema operativo o con funciones de librerías estándar (por ejemplo, perdiendo datos tras una llamada a función).

La siguiente tabla muestra el nombre del registro según la convención de ensamblador (ABI Name), su correspondencia con el registro físico y su función establecida:

Nombre ABI	Registro	Uso Convencional	Preservado
`zero`	x0	Siempre cero (Hardwired)	-
`ra`	x1	Dirección de retorno (*Return Address*)	Sí
`sp`	x2	Puntero de pila (*Stack Pointer*)	Sí
`gp`	x3	Puntero global	Sí
`tp`	x4	Puntero de hilo (*Thread Pointer*)	Sí
`t0 - t2`	x5 - x7	Temporales	No
`s0` / `fp`	x8	Registro salvado / Puntero de marco	Sí
`s1`	x9	Registro salvado	Sí
`a0 - a1`	x10 - x11	Argumentos / Valores de retorno	No
`a2 - a7`	x12 - x17	Argumentos de función	No
`s2 - s11`	x18 - x27	Registros salvados	Sí
`t3 - t6`	x28 - x31	Temporales	No

Memoria:

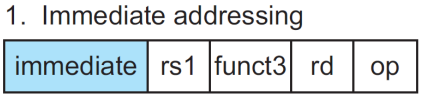
- El acceso a memoria en la variante RV32 se realiza mediante direcciones de 32 bits. Por consiguiente, el mapa de memoria abarca desde 0 hasta $2^{32} - 1$ (0xffffffff).
- El acceso puede realizarse a nivel de byte (8 bits), media palabra (16 bits) y palabra (32 bits).
- Los accesos a palabra (32 bits) se deben realizar a direcciones múltiplos de 4. Los accesos a media palabra (16 bits) serán a direcciones múltiplos de 2.

- La arquitectura RISC-V estándar utiliza el ordenamiento **Little-Endian**. El simulador RARS que utilizaremos en prácticas respeta este formato (el byte menos significativo ocupa la dirección más baja).

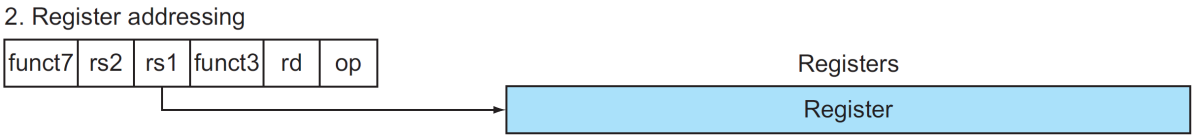
Modos de direccionamiento:

El modo de direccionamiento es la forma en la que se especifica dónde se encuentran cada operando de la instrucción.

- **Inmediato.**
  - El operando se encuentra en la propia instrucción (generalmente **12 bits**).
  - Solo se permite para operandos fuente.
  - En la mayoría de instrucciones (como `addi`, `andi`), se produce una **extensión de signo** del inmediato a 32 bits.
  - Notación: `valor`
  - Ejemplo: `addi t0, s1, -4` $\rightarrow t0 = s1 + (-4)$

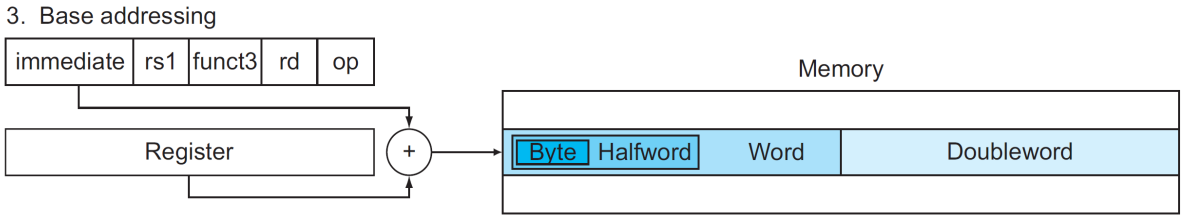


- **Directo a registro.**
  - El operando se encuentra almacenado en un registro ($x0 - x31$).
  - Se permite para operandos fuente o destino.
  - Notación: `registro`
  - Ejemplo: `add s3, s2, s1` $\rightarrow s3 = s2 + s1$

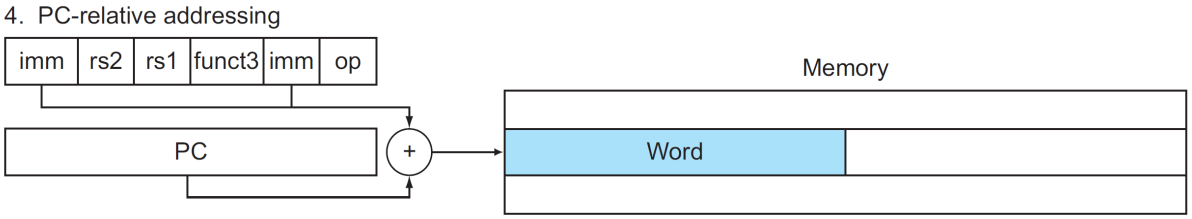


- **Indirecto a registro con desplazamiento.**
  - El operando se encuentra en memoria. La dirección efectiva se calcula sumando un desplazamiento (**12 bits con signo**) al valor del registro base.
  - Se utiliza en cargas (`lw`) y almacenamientos (`sw`).

- Notación: `desplazamiento(registro)`
- Ejemplo: `lw s2, -40(t1)` $\rightarrow s2 = Mem[t1 - 40]$



- **Relativo al PC (Saltos Condicionales).**
  - Se utiliza en instrucciones de salto condicional (tipo B).
  - La dirección destino se calcula sumando el desplazamiento (**12 bits**, extendido en signo) a la dirección del **PC actual**.
  - Ejemplo: `beq t4, t6, etiqueta` $\rightarrow$ Si $t4 == t6$, $PC = PC + offset$



- **Relativo al PC (Saltos Incondicionales).**
  - La instrucción `jal` (tipo J) utiliza direccionamiento relativo al PC.
  - Dispone de un desplazamiento mayor que las instrucciones de salto condicionales(**20 bits**).
  - Ejemplo: `jal x0, etiqueta` $\rightarrow$ Salto incondicional (equivalente a `j`) $PC = PC + offset$.

Formato de instrucciones:

Todas las instrucciones RISC-V (variante RV32) tienen una longitud fija de 32 bits (el mismo tamaño que el de una palabra). A los distintos campos de las instrucciones se les da un nombre para su rápida identificación. El nombre y significado de cada campo en la arquitectura RISC-V se muestra a continuación:

- **opcode (Cód. Op.):** Código de operación de 7 bits (bits 0-6). Determina el formato de la instrucción.

- **rd:** Registro destino de 5 bits(bits 7-11). Es donde se almacena el resultado.
- **funct3:** Campo de función de 3 bits (bits 12-14). Define la sub-operación (ej. ancho de datos o tipo de comparación).
- **rs1:** Primer registro fuente o base de 5 bits (bits 15-19).
- **rs2:** Segundo registro fuente de 5 bits (bits 20-24). Solo presente en formatos R, S y B.
- **funct7:** Campo de función auxiliar de 7 bits (bits 25-31). Distingue operaciones aritméticas similares (ej. `add` vs `sub`).
- **imm:** Valor inmediato. Dependiendo del formato, representa:
  - Constantes aritméticas (12 bits con signo).
  - Desplazamientos de memoria (12 bits).
  - Desplazamientos de salto condicional (12 bits, formato B).
  - Desplazamientos de salto largo o constantes altas (20 bits, formatos J y U).

RISC-V presenta **seis formatos básicos** de instrucción (R, I, S, B, U, J) para optimizar la decodificación hardware.

Tipo	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Ejemplo	Significado
R	funct7	rs2	rs1	funct3	rd	opcode	`add rd, rs1, rs2`	Instrucciones aritméticas
I	imm[11:0]		rs1	funct3	rd	opcode	`addi rd, rs1, 10`	Lecturas de memoria e inmediatos
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode	`sw rs2, 0(rs1)`	Carga de datos en memoria
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode	`beq rs1, rs2, loop`	Saltos condicionales
J	imm[20|10:1|11|19:12]				rd	opcode	`jal rd, func`	Saltos incondicionales
U	imm[31:12]				rd	opcode	`lui rd, 0x1000`	Cálculo con inmediato superior

Clases de instrucciones:

Instrucciones aritméticas y lógicas

- Todas las instrucciones aritméticas y lógicas necesitan que los operandos estén previamente almacenados en registros (*Arquitectura Load-Store*).
- Para cada operación, existen versiones en las que intervienen **3 registros** (`rs1`, `rs2`, `rd`) y versiones con **2 registros y un dato inmediato** (`rs1`, `rd`, `imm`).
- Son instrucciones de formato **Tipo R** (Registro-Registro) o **Tipo I** (Registro-Inmediato).
- En RISC-V, las instrucciones aritméticas básicas de enteros **ignoran el desbordamiento** (*overflow*); no generan excepciones. Además, no existe la instrucción `subi` (resta inmediata); para restar una constante se utiliza `addi` con un valor negativo.

Categoría	Instrucción	Ejemplo	Significado	Comentarios
Aritmética	Suma	`add x5, x6, x7`	$x5 = x6 + x7$	Tres operandos de registro
	Resta	`sub x5, x6, x7`	$x5 = x6 - x7$	Tres operandos de registro
	Suma inmediata	`addi x5, x6, 20`	$x5 = x6 + 20$	Usado para sumar constantes
	1 si es menor que reg	`slt x5, x6, x7`	$x5 = 1$ si $x6 < x7$, sino 0	Tres operandos de registro
	1 si es menor que reg, s. signo	`sltu x5, x6, x7`	$x5 = 1$ si $x6 < x7$, sino 0	Tres operandos de registro
	1 si es menor que inm	`slti x5, x6, 20`	$x5 = 1$ si $x6 < 20$, sino 0	Comparación con inmediato
	1 si es menor que inm, s. signo	`sltiu x5, x6, 20`	$x5 = 1$ si $x6 < 20$, sino 0	Comparación con inmediato
	Multiplicación	`mul x5, x6, x7`	$x5 = x6 \times x7$	32 bits bajos del producto de 64 bits
	Multiplicación alta	`mulh x5, x6, x7`	$x5 = (x6 \times x7) \gg 64$	32 bits altos de 64 bits con signo
	Mult. alta, sin signo	`mulhu x5, x6, x7`	$x5 = (x6 \times x7) \gg 64$	32 bits altos de 64 bits sin signo
	Mult. alta, signo-sin signo	`mulhsu x5, x6, x7`	$x5 = (x6 \times x7) \gg 64$	32 bits altos signo-sin signo
	División	`div x5, x6, x7`	$x5 = x6/x7$	División de núm. de 32 bits con signo
	División sin signo	`divu x5, x6, x7`	$x5 = x6/x7$	División de núm. de 32 bits sin signo
	Resto	`rem x5, x6, x7`	$x5 = x6 \% x7$	Resto de división con signo
	Resto sin signo	`remu x5, x6, x7`	$x5 = x6 \% x7$	Resto de división sin signo
Lógica	And	`and x5, x6, x7`	$x5 = x6 \wedge x7$	Tres op. de reg; AND bit a bit
	Or Inclusivo	`or x5, x6, x8`	$x5 = x6 \vee x8$	Tres op. de reg; OR bit a bit
	Or Exclusivo	`xor x5, x6, x9`	$x5 = x6 \oplus x9$	Tres op. de reg; XOR bit a bit
	And inmediato	`andi x5, x6, 20`	$x5 = x6 \wedge 20$	AND bit a bit reg. con constante
	Or Inclusivo inm.	`ori x5, x6, 20`	$x5 = x6 \vee 20$	OR bit a bit reg. con constante
	Or Exclusivo inm.	`xori x5, x6, 20`	$x5 = x6 \oplus 20$	XOR bit a bit reg. con constante
Desplazam.	Despl. izq. lógico	`sll x5, x6, x7`	$x5 = x6 \ll x7$	Despl. izq. por registro
	Despl. der. lógico	`srl x5, x6, x7`	$x5 = x6 \gg x7$	Despl. der. por registro
	Despl. der. aritmético	`sra x5, x6, x7`	$x5 = x6 \gg x7$	Despl. der. aritmético por registro
	Despl. izq. lóg. inm.	`slli x5, x6, 3`	$x5 = x6 \ll 3$	Despl. izq. por inmediato
	Despl. der. lóg. inm.	`srli x5, x6, 3`	$x5 = x6 \gg 3$	Despl. der. por inmediato
	Despl. der. arit. inm.	`srai x5, x6, 3`	$x5 = x6 \gg 3$	Despl. der. arit. por inmediato

Instrucciones de transferencia y carga / almacenamiento

- **Carga (Load):** Copian información desde la memoria a los registros. Son de formato "Tipo I".
- **Almacenamiento (Store):** Copian información de los registros a la memoria. Son de formato "Tipo S"(el inmediato se divide en dos partes).
- **Transferencia:** Para cargar valores inmediatos en la parte alta (bits 31:12) se utiliza el formato "Tipo U"(`lui`, `auipc`).

Categoría	Instrucción	Ejemplo	Significado	Comentarios
Transf. de datos	Cargar palabra	`lw x5, 40(x6)`	$x5 = M[x6 + 40]$	Palabra de memoria a registro
	Almacenar palabra	`sw x5, 40(x6)`	$M[x6 + 40] = x5$	Palabra de registro a memoria
	Cargar media palabra	`lh x5, 40(x6)`	$x5 = M[x6 + 40]_{16}$	Media palabra de mem a reg
	Cargar media pal, s. sig.	`lhu x5, 40(x6)`	$x5 = M[x6 + 40]_{16}$	Media pal. sin signo de mem a reg
	Almacenar media pal.	`sh x5, 40(x6)`	$M[x6 + 40]_{16} = x5$	Media palabra de reg a mem
	Cargar byte	`lb x5, 40(x6)`	$x5 = M[x6 + 40]_8$	Byte de memoria a registro
	Cargar byte, sin signo	`lbu x5, 40(x6)`	$x5 = M[x6 + 40]_8$	Byte de memoria a registro
	Almacenar byte	`sb x5, 40(x6)`	$M[x6 + 40]_8 = x5$	Byte de registro a memoria
	Cargar inm. superior	`lui x5, 0x12345`	$x5 = 0x12345000$	Carga cte. de 20 bits (desplazada)
	Sumar inm. sup. a PC	`auipc x5, 0x12345`	$x5 = PC + (0x12345000)$	Usado para direcc. relativo a PC

Instrucciones de control de flujo

- **Saltos Condicionales (Branches):** Comparan dos registros y saltan si se cumple la condición. En RISC-V utilizan el formato **Tipo B** (similar al S, pero el inmediato se codifica distinto para ampliar el rango).
- **Saltos Incondicionales (Jumps):**
  - `jal` (*Jump and Link*): Usa el formato **Tipo J** (un registro destino y un inmediato muy grande de 20 bits).
  - `jalr` (*Jump and Link Register*): Usa el formato **Tipo I** (registro destino, registro base e inmediato de 12 bits).

Categoría	Instrucción	Ejemplo	Significado	Comentarios
Salto condicional	Salto si igual	`beq x5, x6, 100`	si ($x5 == x6$) PC+100	Salto relativo a PC si iguales
	Salto si no igual	`bne x5, x6, 100`	si ($x5 \neq x6$) PC+100	Salto relativo a PC si no iguales
	Salto si menor que	`blt x5, x6, 100`	si ($x5 < x6$) PC+100	Salto relativo a PC si menor
	Salto si mayor o igual	`bge x5, x6, 100`	si ($x5 \geq x6$) PC+100	Salto relativo a PC si $\geq$
	Salto si menor, s. signo	`bltu x5, x6, 100`	si ($x5 < x6$) PC+100	Salto relativo a PC si menor (u)
	Salto si mayor o igual, s. signo	`bgeu x5, x6, 100`	si ($x5 \geq x6$) PC+100	Salto relativo a PC si $\geq$ (u)
Saltos	Salto y enlace	`jal x1, 100`	$x1 = PC + 4$; ir a $PC + 100$	Llamada a función. relativa a PC
	Salto y enlace reg	`jalr x1, x5, 100`	$x1 = PC + 4$; ir a $x5 + 100$	Retorno de función.; llamada indirecta

Pseudoinstrucciones

Debido a que el repertorio de instrucciones del **RISC-V** está diseñado para ser lo más reducido posible (siguiendo una filosofía de diseño minimalista), existen operaciones básicas que no se pueden realizar con instrucciones específicas del hardware. Dichas operaciones deben traducirse a una secuencia de instrucciones o realizarse con instrucciones que, originalmente, fueron diseñadas para un fin distinto.

Para facilitar la tarea al programador, el ensamblador define una serie de **pseudoinstrucciones** que realizan dichas operaciones básicas. Las pseudoinstrucciones no forman parte del repertorio de instrucciones de la CPU; es decir, son instrucciones del lenguaje ensamblador que no están implementadas directamente en el hardware.

El ensamblador de RISC-V transforma estas pseudoinstrucciones en instrucciones reales, utilizando frecuentemente el registro $x0$ (denominado *zero*), el cual está cableado internamente para mantener siempre el valor 0. El proceso de conversión transforma una pseudoinstrucción en una o más instrucciones reales del ISA.

Ejemplo de Transferencia de Datos

Un ejemplo de pseudoinstrucción es la siguiente, utilizada para copiar el contenido de un registro a otro:

$$mv \quad x13, x15 \quad \rightarrow \quad x13 = x15$$

El ensamblador transforma esta operación en una instrucción de suma inmediata apoyándose en el registro $x0$:

$$add \quad x13, x15, x0 \quad \rightarrow \quad x13 = x15 + x0$$

Ejemplo de Saltos Condicionales

Muchas de las instrucciones de salto condicional complejo son también pseudoinstrucciones. Por ejemplo, la pseudoinstrucción para saltar si un valor es mayor que otro:

$$bgt \quad x18, x19, \text{loop} \quad \rightarrow \quad \text{Salta a 'loop' si } x18 > x19$$

El ensamblador de RISC-V la traduce simplemente invirtiendo el orden de los operandos y utilizando la instrucción nativa de "saltar si es menor que"(blt):

$$blt \quad x19, x18, \text{loop} \quad \rightarrow \quad \text{Salta a 'loop' si } x19 < x18$$

De esta manera, el hardware se mantiene simple al no tener que implementar circuitos para todas las variaciones de comparación, delegando esa lógica al ensamblador.

Categoría	Instrucción	Ejemplo	Significado	Traducción (Real)
Movimiento de Datos	Copiar registro	`mv x5, x6`	$x5 = x6$	`addi x5, x6, 0`
	Cargar inmediato	`li x5, 123`	$x5 = 123$	`addi x5, x0, 123`
	Cargar dirección	`la x5, label`	$x5 = \&\text{label}$	`auipc` + `addi`
	No operación	`nop`	Nada	`addi x0, x0, 0`
Aritmética y Lógica	Negación arit.	`neg x5, x6`	$x5 = -x6$	`sub x5, x0, x6`
	Negación lógica	`not x5, x6`	$x5 = \sim x6$	`xori x5, x6, -1`
	Set si cero	`seqz x5, x6`	$x5 = (x6 == 0)$	`sltiu x5, x6, 1`
	Set si no cero	`snez x5, x6`	$x5 = (x6 \neq 0)$	`sltu x5, x0, x6`
Saltos Condicionales	Salta si igual 0	`beqz x5, 100`	si $(x5 == 0)$ PC+100	`beq x5, x0, 100`
	Salta si no 0	`bnez x5, 100`	si $(x5 \neq 0)$ PC+100	`bne x5, x0, 100`
	Salta si mayor	`bgt x5, x6, 100`	si $(x5 > x6)$ PC+100	`blt x6, x5, 100`
	Salta si menor	`ble x5, x6, 100`	si $(x5 \leq x6)$ PC+100	`bge x6, x5, 100`
Control de Flujo	Salto incond.	`j 100`	ir a $PC + 100$	`jal x0, 100`
	Retorno	`ret`	$PC = ra$	`jalr x0, ra, 0`
	Llamada func.	`call func`	$ra = PC + 4$; ir a $func$	`auipc` + `jalr`

Las pseudoinstrucciones se crearon para facilitar la vida al programador. Sin embargo, como podemos observar, algunas pseudoinstrucciones se transforman en varias instrucciones reales. Un caso paradigmático es la pseudoinstrucción `li` (Load Immediate), cuya traducción depende del tamaño de la constante que se desea cargar:

- **Caso 1: Constante pequeña (12 bits).** Si el valor cabe en el campo inmediato de una instrucción tipo I, se genera una sola instrucción real.
  Ejemplo: `li x5, 10` → `addi x5, x0, 10`
- **Caso 2: Constante grande (32 bits).** Si el valor es mayor, el ensamblador debe dividirlo en dos partes (parte alta y parte baja), generando dos instrucciones reales.
  Ejemplo: `li x5, 0x12345678` → `lui x5, 0x12345` seguida de `addi x5, x5, 0x678`

Esto supondrá un reto para el alumno en algunas prácticas posteriores, en las que deberá estudiar los programas a partir de las instrucciones reales que los componen (por ejemplo,

para el cálculo del CPI). Por este motivo, el alumno debe conocer cómo se traducen las pseudoinstrucciones que utilice en sus programas.

Directivas del ensamblador

En los programas en ensamblador para RISC-V, es necesario respetar un conjunto de reglas sintácticas fundamentales. Las más habituales son las siguientes:

- Cada línea puede contener como máximo una sentencia (ya sea una instrucción o una directiva).
- Los comentarios comienzan con el carácter `"#"` y abarcan hasta el final de dicha línea.
- Las etiquetas representan una referencia simbólica a una dirección de memoria (donde se encuentran datos o instrucciones); se colocan al principio de la línea y deben finalizar con el carácter `":"`.
- Los valores numéricos pueden expresarse en formato decimal o en hexadecimal (los valores hexadecimales deben comenzar por el prefijo `0x`).
- Las directivas del ensamblador comienzan siempre con el carácter `"."` (punto).

Las directivas sirven para guiar al ensamblador durante el proceso de traducción (por ejemplo, definiendo segmentos de memoria o alineación de datos), pero **nunca generan instrucciones de máquina** ejecutables por la CPU. Las directivas más importantes se detallan a continuación:

Categoría	Directiva	Descripción	Detalles / Notas
Secciones de Memoria	`.text`	Define sección de código	Almacena instrucciones (código máquina).
	`.data`	Define sección de datos	Almacena variables globales.
	`.bss`	Sección BSS	Variables globales inicializadas a 0.
	`.section .foo`	Sección personalizada	Crea una sección llamada `.foo`.
Definición de Datos (Enteros)	`.byte b1..bn`	Almacena 8 bits	Bytes sucesivos.
	`.half w1..wn`	Almacena 16 bits	Halfwords sucesivos.
	`.word w1..wn`	Almacena 32 bits	Words sucesivos.
	`.dword w1..wn`	Almacena 64 bits	Doublewords sucesivos.
	`.string "str"`	Almacena cadena	Cadena de caracteres terminada en `null`.
Definición de Datos (Float)	`.float f1..fn`	Precisión simple (32b)	Almacena floats en words sucesivos.
	`.double d1..dn`	Precisión doble (64b)	Almacena doubles en doublewords sucesivos.
Alineación y Visibilidad	`.align n`	Alineación potencia de 2	Alinear a 2^n bytes (ej. `.align 2` = word).
	`.balign n`	Alineación en bytes	Alinear a n bytes (ej. `.balign 4` = word).
	`.globl sym`	Declarar global	Hace visible la etiqueta `sym` a otros archivos.
Opciones (.option)	`rvc` / `norvc`	Compresión	Habilita/Deshabilita inst. comprimidas (16-bit).
	`relax` / `norelax`	Relajación del Linker	Permite/Prohíbe optimizaciones del linker.
	`pic` / `nopic`	Código indep. posición	Marca instrucciones como PIC o dependientes.
	`push`	Guardar estado	Guarda el estado actual de las opciones en stack.
	`pop`	Restaurar estado	Restaura las opciones desde el stack.

EL SIMULADOR RARS

El simulador RARS es un software que simula el comportamiento de procesadores basados en la arquitectura RISC-V. Para su ejecución, es necesario disponer de una máquina virtual Java 8 o superior, y funciona correctamente tanto en sistemas operativos Windows como en Linux y macOS.

Este simulador no necesita instalación alguna. Lo único necesario es ejecutar el archivo `rars_1_6.jar` con la máquina virtual Java que dispongamos. Esto puede realizarse desde una consola, con el comando:

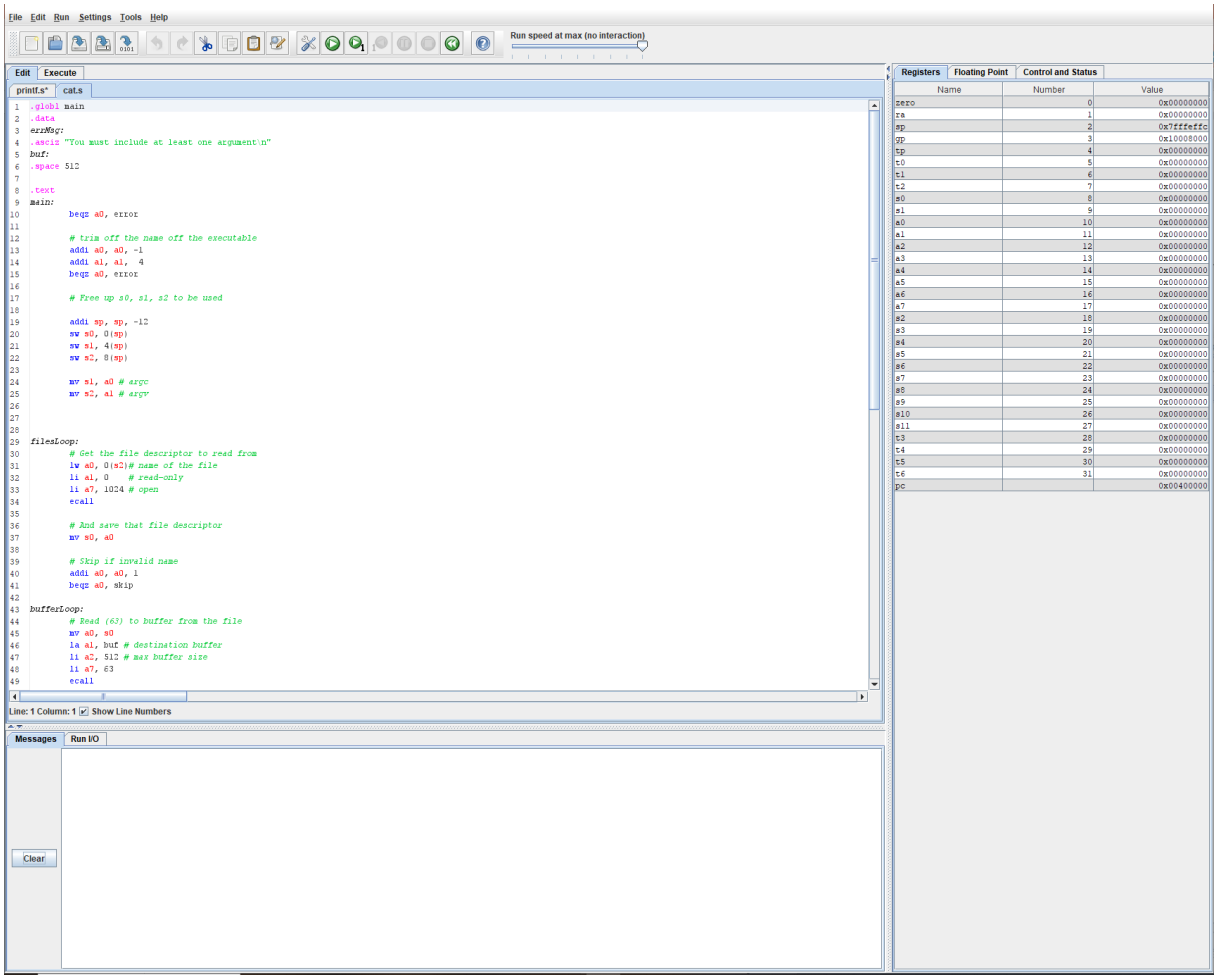
```
java -jar rars_1_6.jar
```

o bien desde la interfaz gráfica del sistema operativo haciendo doble clic sobre el archivo.

Una vez ejecutado, obtendremos una pantalla con la zona de menús, la barra de botones y tres áreas principales:

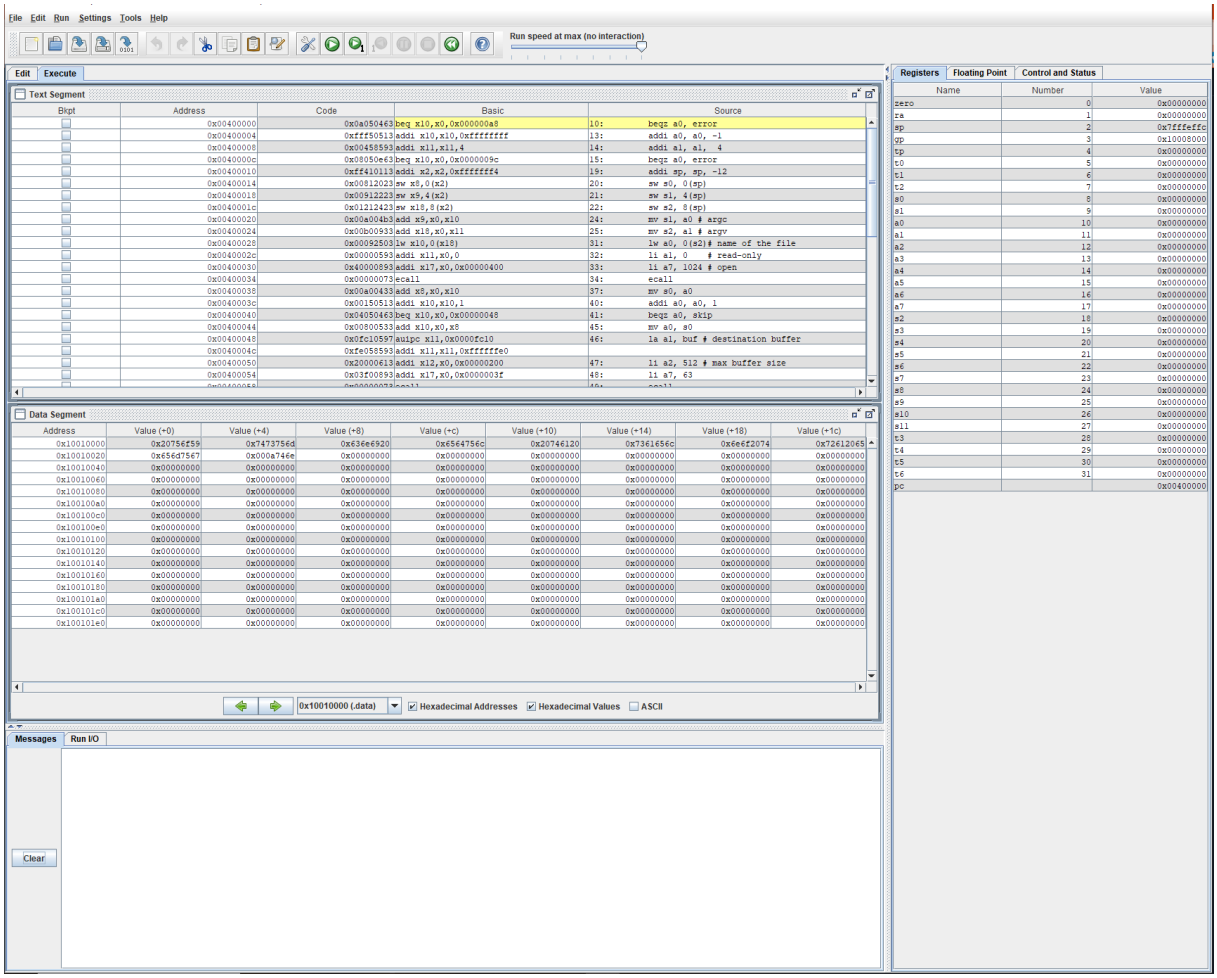
- **La ventana superior izquierda**, donde tenemos dos pestañas: *Edit*, donde se puede editar el código ensamblador del programa, y *Execute*, donde vemos el código ensamblado (Text Segment) y los datos del programa (Data Segment).
- **La ventana inferior**, donde se muestran los mensajes del ensamblador (pestaña *Mars Messages*) y la consola de entrada/salida (pestaña *Run I/O*).
- **La ventana derecha**, donde podemos observar los bancos de registros de la CPU. Esta sección suele organizarse en pestañas para visualizar los registros de enteros (x0-x31), los de punto flotante (f0-f31) y los registros de control y estado (CSR).

Para cargar nuestro programa, seleccionaremos la opción de menú `File` → `Open`. Una vez abierto, veremos el código cargado en la pestaña *Edit*.



Para ensamblar el programa, usaremos indistintamente la opción de menú `Run` → `Assemble`, el botón correspondiente en la barra de herramientas o bien la tecla `F3`.

Una vez ensamblado correctamente el programa, el simulador cambiará automáticamente a la pestaña *Execute*. En ella estarán disponibles las secciones *Text Segment* (donde se visualiza el código máquina generado y las direcciones de memoria) y *Data Segment* (donde se muestra el contenido y las variables en la memoria de datos).



4. Generación de Código Ensamblador desde C

Para analizar cómo se traducen las estructuras de alto nivel a bajo nivel, utilizaremos un compilador cruzado (*cross-compiler*). A diferencia de la compilación nativa, este proceso nos permitirá generar código ensamblador legible para la arquitectura RISC-V (ficheros `.s`) a partir de código fuente en C, en lugar de generar un ejecutable binario para la máquina anfitriona.

A continuación se describen los pasos necesarios para preparar el entorno, crear el código fuente y utilizar el script de automatización.

4.1. Instalación del entorno de compilación (Linux/WSL)

El script de compilación facilitado requiere la presencia del compilador `gcc` específico para la arquitectura RISC-V de 64 bits (que soporta la generación de código para 32 bits mediante flags) y un intérprete de Python para la limpieza del código.

Nota: En los ordenadores de los laboratorios, este entorno ya se encuentra instalado y configurado, por lo que **no es necesario realizar los pasos descritos en esta sección**

a menos que desee configurar su propio equipo personal.

Si necesita preparar un entorno personal basado en Debian/Ubuntu (o WSL en Windows), abra un terminal y ejecute los siguientes comandos para instalar las dependencias:

```
sudo apt update
sudo apt install gcc-riscv64-linux-gnu python3
```

4.2. Creación del fichero fuente en C

Utilice su editor de texto preferido (`nano`, `vim`, `gedit`, `code`, etc.) para escribir el programa en C. Asegúrese de guardar el archivo con la extensión `.c`.

Por ejemplo, para crear un archivo llamado `ejercicio_vectores.c`:

```
nano ejercicio_vectores.c
```

Escriba el código fuente dentro del fichero y guárdelo. Recuerde que, para facilitar el análisis del ensamblador posterior, se recomienda declarar las variables y vectores como globales.

4.3. Uso del script de compilación

Para facilitar el proceso de compilación y la generación del código ensamblador que vamos a ejecutar en el simulador RARS, se proporcionan dos aplicaciones que están disponibles en Enseñanza Virtual junto con este boletín. Se trata de los scripts `compilacion.sh` y `limpia-asm.py`. **Debe descargarlos en la misma carpeta en la que se encuentre el código C que desee compilar.**

El script `compilacion.sh` admite dos modos de funcionamiento: **nativa** (para ejecutar en el PC) y **cruzada** (para generar ensamblador RISC-V).

Cuando se utiliza el modo de compilación **cruzada**, el script ejecutará el compilador gcc para generar el código ensamblador y posteriormente ejecutará el script `limpia-asm.py` para adaptar dicho código a la sintaxis del simulador RARS.

Sintaxis del script:

```
compilacion.sh {nativa|cruzada} fichero_fuente [nivel_opt]
```

Donde:

- **nativa/cruzada**: El modo de compilación. Para esta práctica utilizaremos principalmente `cruzada`.
- **fichero_fuente**: El nombre del archivo `.c` creado en el paso anterior.
- **nivel_opt**: (Opcional) Nivel de optimización (`0`, `1`, `2`, `3`). Por defecto se aplica `0` para evitar que el compilador altere drásticamente la estructura del código.

Pasos para compilar:

Puede compilar su código de dos formas. Se recomienda la primera opción, ya que no requiere modificar los permisos del fichero.

- **Opción A (Recomendada):** Invoque el script directamente con el intérprete `bash`.

  `bash compilacion.sh cruzada ejercicio_vectores.c`

- **Opción B:** Si prefiere ejecutarlo como un programa (`./`), primero debe otorgar permisos de ejecución:

  Primero, otorgue permisos de ejecución al script:

  `chmod +x compilacion.sh`

  A continuación, ejecute la compilación:

  `./compilacion.sh cruzada ejercicio_vectores.c`

El script invocará al compilador `riscv64-linux-gnu-gcc` con los parámetros adecuados (`-march=rv32im -mabi=ilp32`) para generar código compatible con el simulador RARS.

4.4. Análisis del resultado

Si la compilación es exitosa, el script no mostrará el código por pantalla, sino que generará un fichero con código ensamblador que se puede cargar directamente en el simulador RARS. Dicho fichero tendrá el prefijo `rars_` y la extensión `.s`; por ejemplo:

`rars_ejercicio_vectores.s`

Para ver el código ensamblador resultante, puede cargar en RARS el fichero generado. Este fichero contiene las instrucciones RISC-V (como `lw`, `add`, `bge`) que corresponden a la lógica programada en C, listas para ser analizadas o ejecutadas en el simulador.

5. Ejemplo de análisis: Estructura condicional y variables globales

A continuación, se presenta un ejemplo completo de cómo el compilador cruzado traduce un programa en C con variables globales y una estructura de control condicional simple (`if-else`) a lenguaje ensamblador RISC-V.

Este ejemplo ilustra cómo se accede a la memoria estática (segmento de datos) y cómo se traduce la lógica de alto nivel utilizando instrucciones de salto condicional.

5.1. Código Fuente en C

El siguiente programa declara dos variables globales, `x` y `res`. Dependiendo del valor de `x`, se asigna un valor u otro a `res`.

```
int x = 4;
int res = 0;

int main(){
    if(x <= 2){
        res = 20;
```

```
    } else {
        res = 10;
    }
}
```

5.2. Código Ensamblador Completo

A continuación se muestra el código completo resultante tras la compilación y limpieza, listo para ser analizado en el simulador RARS. Nótese que se realiza la gestión de la pila (sp), aunque en este ejemplo las variables principales se manejan en memoria global.

```
    # Segmento de datos
    .data
x:
    .word    4
res:
    .space   4

    # Segmento de código
    .text
.globl main
main:
    # --- Inicio (Gestión de Pila) ---
    addi    sp,sp,-16
    sw      s0,12(sp)
    addi    s0,sp,16

    # --- Condición: if(x <= 2) ---
    la      a5,x            # Cargar dirección de x
    lw      a4,0(a5)        # Cargar valor de x
    li      a5,2            # Cargar constante 2
    bgt     a4,a5,.L2       # Si (x > 2) ir a ELSE (.L2)

    # --- Cuerpo del IF: res = 20 ---
    la      a5,res
    li      a4,20
    sw      a4,0(a5)        # Guardar 20 en res
    j       .L3             # Saltar al final

.L2:
    # --- Cuerpo del ELSE: res = 10 ---
    la      a5,res
    li      a4,10
    sw      a4,0(a5)        # Guardar 10 en res

.L3:
    #  --- Salida ---
    li      a5,0
    mv      a0,a5
```

```
    lw      s0,12(sp)
    addi    sp,sp,16

    li a7, 10               # Syscall Exit
    ecall
```

5.3. Análisis del Código Ensamblador Generado

Al compilar con el script facilitado (`compilacion.sh` usando la compilación cruzada), obtenemos el siguiente código ensamblador. Analizaremos las partes clave.

5.3.1. Segmento de Datos (.data)

Las variables declaradas fuera del `main` son globales. El compilador las ubica en la sección `.data`, no en la pila.

```
    .data
x:
    .word    4          # int x = 4; (Inicializado)
res:
    .space   4          # int res = 0; (Reserva espacio, init a 0)
```

5.3.2. Carga y Comparación

Para evaluar `if(x <= 2)`, el procesador no puede operar directamente sobre la memoria. Debe seguir la secuencia **Cargar Dirección** → **Cargar Valor** → **Comparar**.

```
    # Cargar el valor de 'x' desde memoria
    la     a5, x        # Carga la dirección de memoria de 'x' en a5
    lw     a4, 0(a5)    # Carga el valor contenido en esa dirección
   en a4

    # Preparar la constante de comparación
    li     a5, 2        # Carga el inmediato 2 en a5
```

5.3.3. Lógica de Salto

Aquí ocurre una optimización común. Aunque el C dice `<=`, el compilador utiliza la instrucción inversa `bgt` (Branch if Greater Than). La lógica es: *"Si x es mayor que 2, salta directamente al bloque ELSE (.L2)"*.

```
    # if (x <= 2) se traduce como: ¿Es x > 2?
    bgt    a4, a5, .L2 # Si a4 > a5, salta a la etiqueta .L2 (Else)

    # --- Bloque IF (res = 20) ---
    la     a5, res      # Carga dirección de 'res'
    li     a4, 20       # Carga valor 20
    sw     a4, 0(a5)    # Guarda 20 en memoria
```

```
    j      .L3          # Salta incondicionalmente al final (evita el
    else)
```

5.3.4. Bloque Else y Finalización

Si la condición del salto anterior se cumplió, el flujo llega a la etiqueta `.L2`.

```
.L2: # --- Bloque ELSE (res = 10) ---
    la     a5, res      # Carga dirección de 'res'
    li     a4, 10       # Carga valor 10
    sw     a4, 0(a5)    # Guarda 10 en memoria

.L3: # --- Fin del condicional ---
    li     a5, 0        # Return 0
```

6. Ejercicios

Nota: Debido al formato del documento, es necesario revisar y corregir el código copiado de los ejercicios, dado que se añaden ciertos espacios en blanco que producen errores a la hora de ensamblarlo en el RARS.

***:** Utilizar el compilador para resolver los ejercicios marcados.

Ejercicio 1

A continuación se presenta un programa en ensamblador. Este programa muestra la estructura típica de todo programa en este entorno, que consta de:

- Una sección con los datos del programa (encabezada por la directiva `.data`), que serán almacenados en el área de memoria reservada para datos.
- Una sección con el código del programa (encabezada por la directiva `.text`), que será almacenada en la zona de memoria reservada para el código. El programa principal se referencia siempre por la etiqueta `main`.

```
# Practicas AC P1-EJ1
.data
datos: .space 32

.text
main:   la t2, datos          # Carga la direccion de 'datos' en t2
        lui t1, 0x88990       # Carga inmediato superior
        sw t1, 16(t2)         # Almacena la palabra de t1 en memoria
        lb t1, 19(t2)         # Carga el byte del offset 19 a t1

final:  li a7, 10             # Codigo de servicio 10 (exit)
        ecall                 # Llamada al sistema para finalizar
```

Las dos últimas instrucciones, "`li a7, 10`" y "`ecall`", sirven para terminar la ejecución del programa.

Actividades

1. Analice el código teniendo en cuenta el funcionamiento de las instrucciones. ¿Qué valor tendrá el registro `t1` al final de la ejecución del programa en ensamblador?

2. Abra el simulador RARS y genere un nuevo archivo de programa (`File` → `New`). Escriba el código anterior (o utilice Copy & Paste) y guárdelo con la extensión ".asm". Ensamble el código utilizando `Run` → `Assemble`.

   En la ventana de código (*Execute*), observe las columnas:

   - **Address:** Contiene la dirección de memoria en que se almacena cada instrucción.
   - **Code:** Contiene la traducción a código máquina de la instrucción.
   - **Basic:** Contiene las instrucciones reales a las que ha dado lugar el código en ensamblador, con las etiquetas traducidas a direcciones y las pseudoinstrucciones expandidas.
   - **Source:** Contiene el código ensamblador original del programa.

3. Ejecute el programa: `Run` → `Go`. Si no hay ningún error, aparecerá el mensaje indicando que el programa ha finalizado en la consola inferior.

4. Teniendo en cuenta que el simulador utiliza el ordenamiento **little-endian**, conteste a las dos preguntas siguientes sin ejecutar el código; después, verifique sus respuestas con el simulador:

   - ¿Cuál será el valor del registro `t1` después de ejecutar la instrucción `lb`?
   - ¿Y si se ejecuta la instrucción `lbu` (Load Byte Unsigned) en vez de `lb`?

   NOTA: Si no consigue averiguar la respuesta, ejecute el código paso a paso y busque una explicación para los resultados obtenidos en el registro.

5. ¿Cómo se ha traducido la pseudoinstrucción `la`?

Control de flujo: estructuras condicionales

Ejercicio 2

Dado el siguiente fragmento de código en lenguaje C:

```
r = 0;
a = 1;
b = 2;
if (a <= b)
    r = 1;
```

A continuación se presenta un programa equivalente en ensamblador para RISC-V.

```
# Practicas AC P1-EJ2
.data
#*** Comienzo de la seccion que se almacena en el area de datos
# (Contenido: datos)

.text
#*** Comienzo de la seccion que se almacena en el area de texto
# (Contenido: codigo)

main:   #*** Comienzo del programa principal
        addi t1, x0, 0      # r = 0
        addi t2, x0, 1      # a = 1
        addi t3, x0, 2      # b = 2

        bgt t2, t3, continua # Salta a continua si a > b
        addi t1, x0, 1       # r = 1

continua:
        li a7, 10            # Llamada al sistema para finalizar
        ecall
```

Una estructura condicional de los lenguajes de alto nivel se traduce en lenguaje ensamblador utilizando instrucciones de salto condicional. RISC-V dispone de un conjunto de instrucciones nativas para comparar y saltar en un solo paso (como `beq`, `bne`, `blt`, `bge`).

Sin embargo, para facilitar la programación, el ensamblador ofrece pseudoinstrucciones adicionales que no existen físicamente en el hardware, como `bgt` (Branch if Greater Than) o `ble` (Branch if Less or Equal).

A diferencia de otras arquitecturas antiguas que requerían múltiples instrucciones para simular esto, RISC-V resuelve estas pseudoinstrucciones de forma inteligente: simplemente intercambia el orden de los operandos y utiliza la instrucción nativa opuesta. Por ejemplo, la condición $a > b$ es lógicamente equivalente a $b < a$, por lo que el ensamblador traduce un `bgt` a un `blt` invirtiendo los registros.

Obsérvese también cómo la comparación que se realiza en el código ensamblador (`bgt`, si $a > b$) es la negada de la condición original del `if` en C ($a \leq b$). Esto se hace para saltar por encima de la asignación `r=1` cuando la condición no se cumple.

Actividades

1. Estudie el código de ambos programas (en lenguaje C y en ensamblador). ¿Qué valor tendrá el registro `t1` al final de la ejecución del programa en ensamblador?

2. Abra RARS y genere un nuevo archivo de programa (`File` → `New`). Escriba el código anterior (o utilice Copy & Paste) y guárdelo con la extensión ".asm". Ensamble el código utilizando `Run` → `Assemble`.

3. Ejecute el programa: `Run` → `Go`. Si no hay ningún error, aparecerá el mensaje indicando que el programa ha finalizado.
4. Estudie el contenido de los registros que usa el programa tras la ejecución del mismo. En concreto, el registro `t1`, ¿contiene el valor esperado tras ejecutarse el programa?
5. Examine con detenimiento el programa y su conversión a instrucciones reales.
   - ¿Cuántas pseudoinstrucciones tiene el programa?
   - ¿Cómo se ha traducido exactamente la pseudoinstrucción `bgt`? (Fíjese en la instrucción real generada y en el orden de los registros `t2` y `t3`).
   - ¿Cuántas instrucciones reales tiene finalmente el código máquina del programa?

Ejercicio 3

Escriba un programa en ensamblador para RISC-V equivalente a la siguiente porción de código en lenguaje de alto nivel (C):

```
i = 15;
if (i < 10)
    r = 0;
else if (i < 20)
    r = 1;
else
    r = 2;
```

NOTA: Al final del programa en ensamblador añada una única llamada al sistema para finalizar.

Actividades

- Ejecute el programa en el simulador RARS y compruebe si, tras la ejecución, el valor del registro equivalente a la variable `r` tiene el valor adecuado.
- Ejecute el programa cambiando el valor que toma `i` al principio (equivalente a la primera instrucción del código en C), probando los siguientes valores: `i = 5`, `i = 25`. Compruebe en cada caso que tras la ejecución el valor del registro equivalente a la variable `r` tiene el valor adecuado.

Ejercicio 4 *

Cree un fichero llamado `condicional.c` con el siguiente código C. Este programa determina si un número es par o impar utilizando el operador módulo.

```
int numero = 7;
int es_par = 0;

void main() {
    if (numero % 2 == 0) {
```

```
        es_par = 1;
    } else {
        es_par = 0;
    }
}
```

Actividades

1. Compile el código utilizando el script en modo cruzada:
   `bash compilacion.sh cruzada condicional.c`
2. Abra el fichero generado `rars_condicional.s` y localice la traducción de la instrucción `if`.
3. Responda a las siguientes cuestiones:
   - ¿Qué instrucción de ensamblador utiliza el compilador para realizar la operación módulo (`%`)?
   - Observe la instrucción de salto condicional (probablemente `bne` o `beq`). ¿La lógica del salto es la misma que en C o está invertida? (Ej: ¿Salta si es verdadero o salta si es falso para ir al `else`?)

Control de flujo: estructuras iterativas

Ejercicio 5

Las estructuras iterativas de los lenguajes de alto nivel se traducen usando una instrucción de salto condicional (que puede ir precedida de una instrucción `slt`) para implementar la condición del bucle, además de una instrucción de salto incondicional para volver al comienzo del bucle y ejecutar la siguiente iteración.

Como ejemplo, vamos a traducir la siguiente porción de código en C:

```
x = 4;
res = 0;
for (i = 0; i < x; ++i)
    res = res + 3;
```

Un programa equivalente en ensamblador del RISC-V es:

```
# Practicas AC P1-EJ5
.data

.text
main:
    addi t1, zero, 4     # x = 4
    addi t2, zero, 0     # res = 0
    addi t3, zero, 0     # i = 0

bucle:
    slt t4, t3, t1       # i < x ?
```

```
    beq t4, zero, fin    # si i >= x, fin
    addi t2, t2, 3       # res = res + 3
    addi t3, t3, 1       # i++
    j bucle              # Salto incondicional al inicio

fin:
    li a7, 10            # Llamada al sistema para finalizar
    ecall
```

Actividades

- Ejecute el programa en el simulador RARS y compruebe si tras la ejecución el valor del registro equivalente a la variable `res` tiene el valor adecuado.

Ejercicio 6

Escriba un programa en ensamblador para RISC-V equivalente a la siguiente porción de código en lenguaje de alto nivel (lenguaje C):

```
x = 4;
res = 0;
i = 1;
while (i <= x) {
    res = res + 2;
    ++i;
}
```

Actividades

- Ejecute el programa en el simulador RARS y compruebe si tras su ejecución el valor del registro equivalente a la variable `res` tiene el valor adecuado.

Ejercicio 7 *

Cree un fichero llamado `bucle.c`. Este programa realiza una suma acumulativa de los primeros 5 números naturales.

```
int total = 0;
int i;
void main() {
    for (i = 1; i <= 5; i++) {
        total = total + i;
    }
}
```

Actividades

1. Compile el código en modo cruzada.

2. Analice el cuerpo del bucle dentro del fichero ensamblador generado.
3. Responda a las siguientes cuestiones:
   - Cuente cuántas instrucciones de carga (`lw`) y almacenamiento (`sw`) se ejecutan **dentro** de cada iteración del bucle.
   - Considerando que el código se ha compilado sin optimización (`-O0`), ¿por qué cree que hay tantos accesos a memoria en lugar de mantener los valores en los registros?
   - Identifique las etiquetas que delimitan el inicio y el fin del bucle.

Procesamiento de Vectores

Ejercicio 8

Consideremos el siguiente programa en lenguaje de alto nivel (lenguaje C), que suma dos vectores o arrays guardando el resultado en un tercer vector y además acumula la suma de todos los elementos en una variable:

```
// Practicas AC P1-EJ8
int n = 3, suma = 0;
int a[3] = {1, 3, 5};
int b[3] = {2, 4, 6};
int r[3];
main() {
    int i;
    for (i = 0; i < n; ++i) {
        r[i] = a[i] + b[i];
        suma += r[i];
    }
}
```

A continuación se presenta un programa equivalente en ensamblador del RISC-V:

```
# Prácticas AC P1-EJ8
.data
# variables globales n (tamaño de los arrays) y suma:
n:    .word 3
suma: .space 4
# arrays a, b y r:
a:    .word 1, 3, 5
b:    .word 2, 4, 6
r:    .space 12

.text
main:
    # vamos a cargar en t1 el tamaño de los arrays (n)
    la t1, n
    lw t1, 0(t1)
    # vamos a cargar en t2 la dirección base del array a
    la t2, a
    # vamos a cargar en t3 la dirección base del array b
    la t3, b
    # vamos a cargar en t4 la dirección base del array r
    la t4, r

    addi t5, zero, 0    # suma_reg = 0
    addi t6, zero, 0    # i = 0

bucle:
    # Usamos t0 para la condicion y s1, s2 como temporales extra.
```

```
    slt t0, t6, t1        # i < n ?
    beq t0, zero, fin     # si i >= n, fin

    lw s1, 0(t2)          # a[i]
    lw s2, 0(t3)          # b[i]
    add s1, s1, s2
    sw s1, 0(t4)          # r[i] <-- a[i] + b[i]

    add t5, t5, s1        # suma_reg = suma_reg + a[i] + b[i]

    addi t2, t2, 4        # incrementa el puntero del array a
    addi t3, t3, 4        # incrementa el puntero del array b
    addi t4, t4, 4        # incrementa el puntero del array r
    addi t6, t6, 1        # ++i
    j bucle

fin:
    # vamos a cargar en t1 la dirección de la variable global suma
    la t1, suma
    sw t5, 0(t1)          # suma <-- suma_reg

    li a7, 10             # Llamada al sistema para finalizar
    ecall
```

Actividades

- ¿Qué valores deben tener el array `r` y la variable `suma` del programa en C tras su ejecución?
- Estudie atentamente cada una de las instrucciones del programa en ensamblador asegurándose de entender su significado. Observe las técnicas usadas para minimizar el número de registros usados.
- Ejecute el programa en el simulador RARS y compruebe si tras la ejecución los valores adecuados de las variables globales aparecen cargados en memoria. Compruebe también si los valores cargados en los registros usados son los adecuados.

EJERCICIOS ADICIONALES

Ejercicio 9

Realice la traducción del siguiente código en C a lenguaje ensamblador RISC-V.

El objetivo es practicar cómo se traducen las llamadas a una función desde otra función; se deben pasar los parámetros correctamente a la función utilizando los registros de argumentos (`a0`, `a1`, ...) y recoger el resultado de vuelta.

Código C:

```c
int resultado = 0;

int suma(int a, int b) {
    return a + b;
}

void main() {
    int val1 = 10;
    int val2 = 5;

    // Llamada a la funcion
    resultado = suma(val1, val2);
}
```

Plantilla para el código ensamblador: Complete el código prestando atención a cómo se deben cargar los valores antes de llamar a la función con `jal`.

```
# Practica AC P1 - EJ9
.data
resultado: .word 0

.text
.globl main

# --- Rutina Principal ---
main:
    # 1. Cargar los valores 10 y 5 en registros temporales (ej: t0,
    t1)

    # 2. Preparar los argumentos para la funcion:
    #    Mover el primer valor al registro a0
    #    Mover el segundo valor al registro a1

    # 3. Llamar a la funcion 'suma' (usar instruccion jal)

    # 4. Al volver, el resultado estara en a0.
    #    Guardar a0 en la variable global 'resultado'

    # Fin del programa
    li a7, 10
    ecall

# --- Rutina de la Funcion ---
suma:

    # 1. Sumar los argumentos (a0 y a1) y dejar el resultado en a0

    # 2. Retornar a quien nos llamo (usar instruccion ret o jr ra)
```

Ejercicio 10 *

Cree un fichero llamado `funciones.c`. Este ejemplo muestra una función `main` que llama a una función auxiliar `suma`.

```
int resultado = 0;

int suma(int a, int b) {
    return a + b;
}

void main() {
    int x = 10;
    int y = 5;
    resultado = suma(x, y);
}
```

Actividades

1. Compile el código y abra el fichero `.s` resultante.
2. Busque la etiqueta `suma:` y analice las primeras instrucciones y las últimas.
3. Responda a las siguientes cuestiones:
   - Identifique las instrucciones que modifican el puntero de pila (`sp`). ¿Cuántos bytes reserva la función `suma` en la pila?
   - ¿En qué registros se pasan los argumentos `x` e `y` justo antes de llamar a la función con la instrucción `jal`?
   - ¿En qué registro deposita la función `suma` el resultado antes de retornar?

Ejercicio 11

Dado un vector de enteros de 32 bits que se encuentra en la dirección `Valores`, escriba un programa que busque todos los valores negativos y los almacene en otro vector (que se encuentra en dirección `Negativos`). El número de elementos del vector se encuentra en la dirección `Tam`.

a) Escriba el programa utilizando un bucle que recorra el vector `Valores` comenzando por el primer elemento.

b) Escriba el programa utilizando un bucle que recorra el vector `Valores` comenzando por el último elemento (indicado por `Tam`).

Utilice la siguiente plantilla para realizar los dos apartados. Pruebe el programa con diferentes valores de `Tam`.

```
#Practica AC P1-EJ11
.data
Tam:        .word 10
Vector:     .word 0, 3, -2, -3, 10, -4, 23, -5, -11, -10
```

```
Negativos: .space 40

.text
main:

fin:
    li a7, 10
    ecall
```

Ejercicio 12

Implemente las siguientes pseudoinstrucciones utilizando las instrucciones reales. Realice los ejercicios en papel y, posteriormente, use el simulador RARS para comprobar que la solución es correcta. Para realizar dicha comprobación deberá inicializar los registros con valores adecuados.

a) Implemente la pseudoinstrucción "`neg t1, t2`" con una sola instrucción real. Esta pseudoinstrucción asigna a `t1` el valor `-t2`.

b) Implemente la pseudoinstrucción "`not t1, t2`" con una sola instrucción real. Esta pseudoinstrucción asigna a `t1` la negación lógica bit a bit del valor de `t2`.

c) Implemente la pseudoinstrucción "`sge t1, s1, s2`" con dos instrucciones reales. Esta pseudoinstrucción asigna a `t1` el valor 1 si `s1 >= s2`, en caso contrario le asigna un 0. Pruebe su solución con los siguientes valores:

s1	**s2**
2	0
2	2
2	3

d) Implemente la pseudoinstrucción "`sle t1, s1, s2`" con dos instrucciones reales. Esta pseudoinstrucción asigna a `t1` el valor 1 si `s1 <= s2`, en caso contrario le asigna un 0. Pruebe su solución con los valores anteriores.

Ejercicio 13

Escriba una secuencia de instrucciones que convierta una cadena de caracteres en mayúscula a minúscula. La cadena comienza en la dirección a la que apunta la etiqueta `cadena` y el valor 0 indica fin de cadena. Los caracteres de la 'a' a la 'z', tanto en minúscula como en mayúscula, tienen códigos ASCII consecutivos ('A' es el 65 y 'a' el 97). Para convertir un carácter en mayúscula 'Q' a minúscula hay que calcular, por tanto, 'Q'-65+97.

```
#Prácticas AC P1-EJ13
.data
cadena: .asciiz "HOLA RISC-V"

.text
main:
```

```
FIN:
    li a7, 10          # Llamada al sistema para finalizar
    ecall
```

Ejercicio 14

Escriba un programa que calcule el factorial del valor almacenado en el registro `t1`. El resultado debe almacenarse en el registro `t2`.

Ejercicio 15

Escriba un programa que busque en un vector de 16 enteros de tamaño byte el mayor y el menor valor. El vector se encuentra en la dirección `Vector` y los valores encontrados deben almacenarse en las direcciones `Mayor` y `Menor`. Utilice la siguiente plantilla:

```
#Practica AC P1-EJ15
.data
Vector: .byte 0x41, 0x5, 0x69, 0xFE, 0x0F, 0x28, 0xEF, 0x11
        .byte 0x22, 0x3, 0x25, 0xE4, 0x77, 0x0A, 0x78, 0x15
Mayor:  .byte 0
Menor:  .byte 0

.text
main:

fin:
    li a7, 10
    ecall
```

Ejercicio 16

A partir del código del ejercicio 14, escriba un programa que contenga una rutina llamada `factorial` que calcule el factorial del valor almacenado en el registro `a0` y que devuelva el resultado en el registro `a0` (en RISC-V el registro de retorno estándar es a0, equivalente al v0 de otras arquitecturas). El programa debe almacenar en `t1` el factorial de 8 y en `t2` el factorial de 11. Por lo tanto, el programa debe llamar dos veces a la rutina.

NOTA: Utilice la instrucción `jal` para llamar a la rutina y la instrucción `jalr` (o la pseudoinstrucción `ret`) con el registro `ra` para retornar al punto de llamada.